COMP 2233: DATA STRUCTURES

Single Dimension Arrays in C

Problem Solving, Memory Architecture, and Core Array Operations

Assoc.Prof.Dr. Ömer ÇETİN

Problem Solving with Computer Pipeline

From Real-World Problem to Executable Hardware Program

- Human Level: Problem understanding and Algorithm Development.
- Software Level: Program Development (writing C/C++ source code) and Program Translation (Compiler/Assembler).
- Hardware Level: Executing Machine Code and generating Outputs.
- Feedback Loops: Syntax errors caught during translation; Semantic/logical errors caught during execution.

1. Algorithm Dev.

Human conceptualizes step-by-step logic

2. Program Dev.

Human writes source code in C/C++

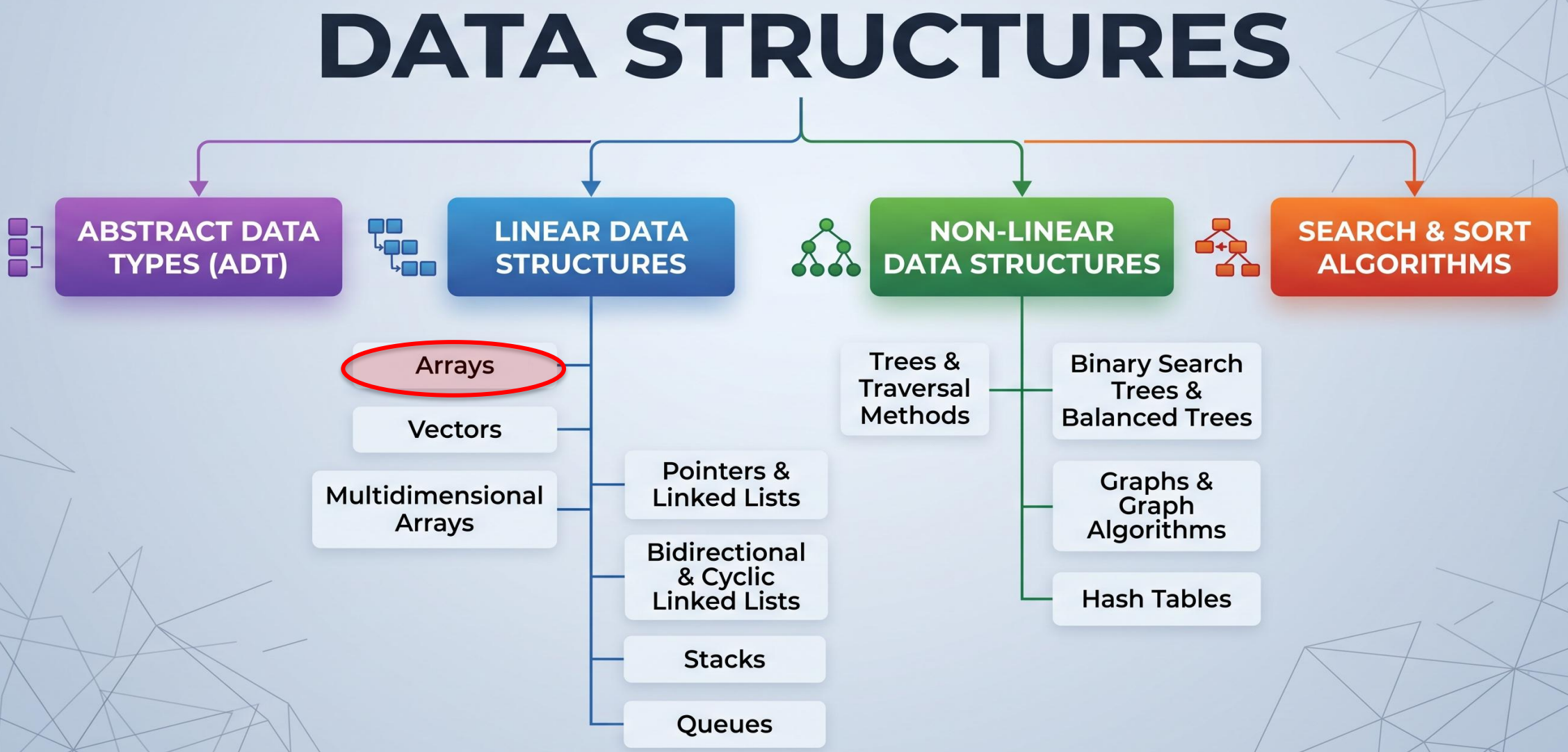
3. Translation

Compiler translates source code to binary

4. Execution

Hardware CPU executes binary instructions

Course Content



What is an Array? Definition & Characteristics

Fundamental Definition

- Homogeneous Collection: Sequence of elements sharing the exact same data type.
- Contiguous Memory: Stored in uninterrupted, adjacent memory locations.
- Random Access: Any element can be directly accessed in O(1) constant time via index.
- Efficient Storage: Ideal when element count is fixed and rapid lookup is required.

Key Characteristics

- Fixed Size: Capacity is specified at declaration and remains static throughout runtime.
- Zero-Based Indexing: Indexing begins at 0 (first element = index 0, last = N-1).
- Type Safety: All elements enforce identical data type (e.g., all `int` or all `float`).
- O(1) Direct Access: Physical address derived instantly via mathematical offset.

Static Memory Layout & Loop Operations

Memory Layout Principles

- Static Elements: Array size is fixed during program compilation/execution.
- Loop Compatibility: For loops are perfectly suited for iterating through arrays sequentially.
- Physical Memory Offsets: RAM allocates adjacent slots (e.g., addresses 201, 202, 203...).

Physical Address

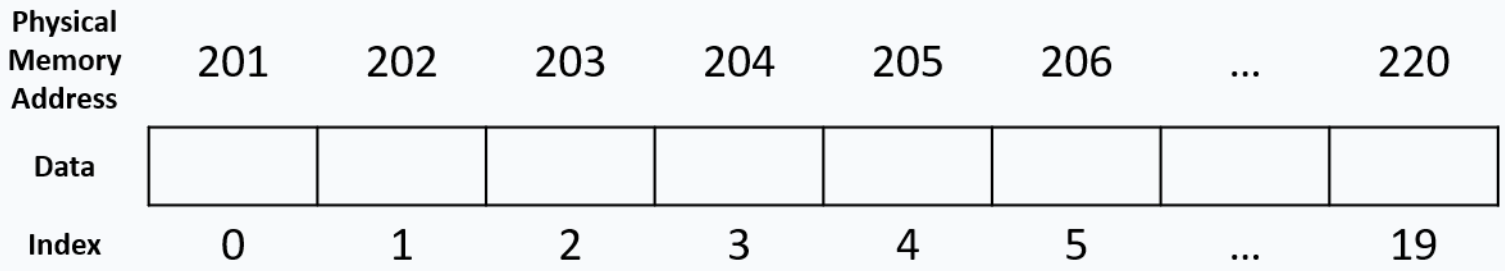
RAM Locations: 201, 202, 203, 204, 205 ... 220

Data Elements

Contiguous Byte Block storing homogeneous values

Zero-Based Index

Sequential Index Offsets: 0, 1, 2, 3, 4, 5 ... 19



Shared Identifier & Direct Access

Shared Identifier Principle

- Common Identifier: All array elements are accessed using a single identifier name.
- Sequential Space: Memory manager reserves a single continuous chunk of bytes.
- Distinguishing Feature: Elements are distinguished solely by their offset index.
- Variable Consolidation: Replaces N individual variable declarations with one array.

Direct Memory Access O(1)

- Direct Retrieval: No need to traverse prior elements to reach index i.
- Mathematical Formula: `Address(arr[i]) = Base_Address + (i * sizeof(DataType))`.
- Execution Speed: Single CPU arithmetic instruction yields instantaneous lookup.
- Constant Time Bounds: O(1) lookup regardless of whether array has 10 or 10,000 items.

Array Classification: 1D vs. Multi-Dimensional

Dimension Taxonomy Overview

- 1-Dimensional (1D) Arrays: Linear sequence of elements accessed by a single index `arr[i]`.
- 2-Dimensional (2D) Arrays: Grid/Matrix structure of rows and columns accessed by `arr[row][col]`.
- N-Dimensional Arrays: Higher-order tensors (e.g., 3D cube buffers in graphics/physics).

Single D. Index

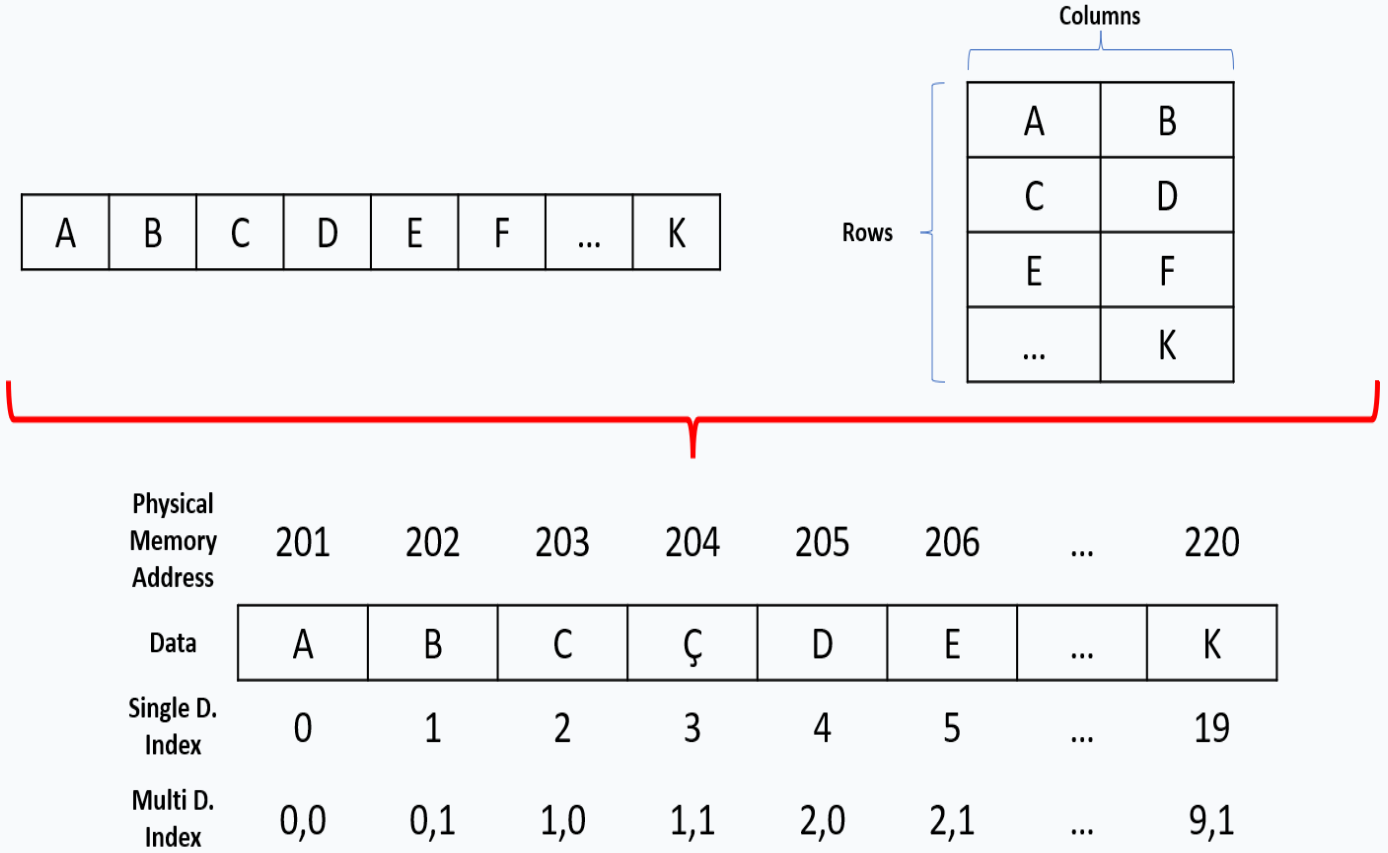
Linear Array: [A, B, C, D, E, F ... K] -> Indexes: 0, 1, 2, 3...

Multi D. Index

Matrix Grid (2 Rows x 3 Cols): (0,0) (0,1) (1,0) (1,1)...

Physical RAM Mapping

All dimensions are flattened into a single 1D RAM line



1D Array Syntax & Declaration Rules

Syntax Patterns in C/C++

- Declaration: `data_type array_name[size];` (e.g., `int Numbers[5];`).
- Declaration + Initialization: `int Numbers[5] = {11, 5, 23, 4, 54};`.
- Value Assignment: `array_name[index] = value;` (e.g., `Numbers[2] = 77;`).
- Value Access: `int x = array_name[index];` (e.g., `int a = Numbers[1];`).
- Subscript Operator `[]`: Encloses index integer offset.

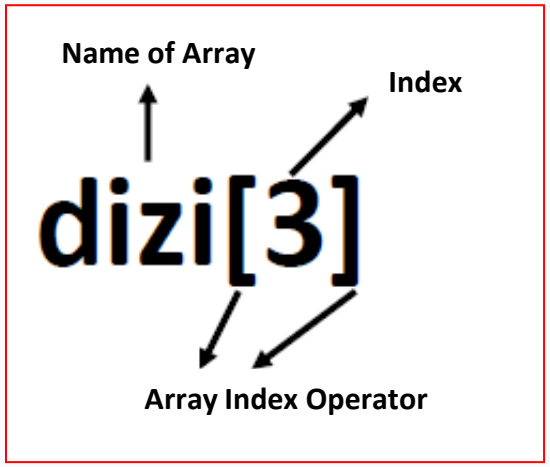
```c
// Array Declaration & Mutation Code
#include <stdio.h>

int main(void) {
    // 1. Declaration & Initialization
    int Numbers[5] = {11, 5, 23, 4, 54};

    // 2. Assignment / Mutation
    Numbers[2] = 77; // Replaces 23 with 77 at index 2

    // 3. Accessing Element
    int a = Numbers[1]; // Retrieves 5 from index 1

    printf("Numbers[1] = %d | Numbers[2] = %d\n", a, Numbers[2]);
    return 0;
}
```



Zero-Based Subindexing Rules & Examples

Indexing Boundary Formulas

- First Element Index: Always `0`.
- Second Element Index: Always `1`.
- N-th Element Index: Always `n - 1`.
- Last Element Index: Always `length - 1`.
- Example Array: `int scores[4] = {97, 86, 92, 71};`.

Index 0

Value: 97 (First Element)

Index 1

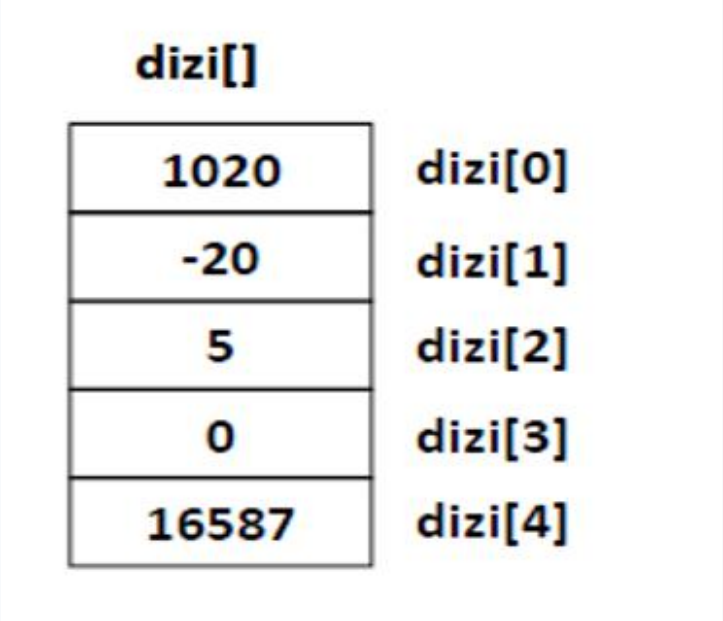
Value: 86 (Second Element)

Index 2

Value: 92 (Third Element)

Index 3

Value: 71 (Last Element = Length 4 - 1)



Sizing Constraints & Partial Initialization

Initialization Rules in C

- Unknown Size Error: Defining `int Numbers[];` without initializers triggers compile error.
- Fixed Runtime Bound: Array capacity cannot be expanded or shrunk during program run.
- Full Initialization: `int n[5] = {2, 12, 13, 4, 5};` populates all 5 slots.
- Partial Initialization: `int n[5] = {10, 20};` populates first 2 slots; remaining set to 0.
- Zero Initialization: `int n[5] = {0};` sets all 5 slots to zero.

```
int[4] scores = {97, 86, 92, 71};
```

index:	0	1	2	3
value:	97	86	92	71

```
// Initialization Code Variations
#include <stdio.h>

int main(void) {
    // Exact initialization
    int a[5] = {2, 12, 13, 4, 5};

    // Partial initialization (rest become 0)
    int b[5] = {10, 20}; // b becomes {10, 20, 0, 0, 0}

    // Zero-fill initialization
    int c[5] = {0};      // c becomes {0, 0, 0, 0, 0}

    printf("b[2] = %d | c[4] = %d\n", b[2], c[4]);
    return 0;
}
```

Calculating Size & Out-of-Bounds Hazards

Element Count Formula & C Safety Hazard

- Calculating Element Count: `int count = sizeof(arr) / sizeof(arr[0]);`.
- No Runtime Bounds Checking: C and C++ do NOT throw runtime errors for out-of-bounds indexes!
- Undefined Behavior: Reading/writing `arr[10]` on a size-5 array corrupts adjacent RAM variables.
- Best Practice: Always maintain size variables or constants and validate loop bounds.

```c
// Size Calculation Code
#include <stdio.h>

int main(void) {
    int Numbers[5] = {11, 5, 23, 4, 54};

    // Total Bytes / Byte size of one element
    int count = sizeof(Numbers) / sizeof(Numbers[0]);

    printf("Total Array Bytes: %zu\n", sizeof(Numbers));
    printf("Number of Elements: %d\n", count);

    // HAZARD: C permits accessing Numbers[10] (Garbage/Crash!)
    return 0;
}
```

The Cost of Element Shifting in Arrays

Insertion & Deletion Performance Bottleneck

- Sorted Array Insertion: Inserting 'Ç' between 'C' and 'D' requires shifting 'D', 'E', 'F', 'G' one slot to the right.
- Sequential Copy Cost: Each shifted item requires a read-write RAM operation.
- Time Complexity: O(N) linear time worst-case for middle insertions and deletions.
- Why Shift?: To preserve contiguous memory layout and zero-gap indexing.

Initial Sorted Array

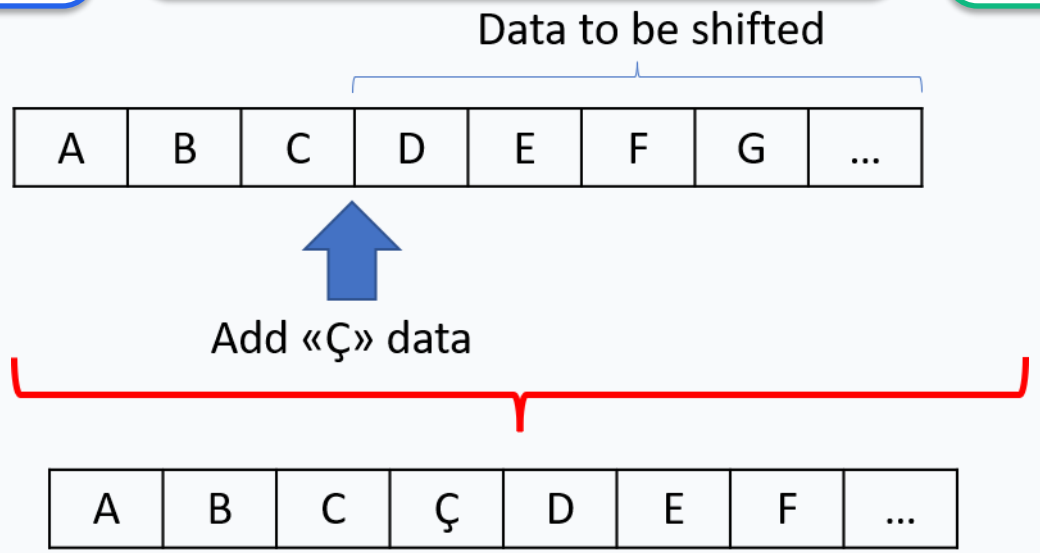
[A | B | C | D | E | F | G]

Right Shift Items (D..G)

[A | B | C | _ | D | E | F | G]

Insert 'Ç' at Vacated Index

[A | B | C | Ç | D | E | F | G]



Anatomy of an Array Expression: `sicaklik[n + 2] = 32;`

Deconstructing Expression Components

- Array Name (`sicaklik`): Acts as pointer to base address of the array in RAM.
- Index Expression (`[n + 2]`): Calculated integer expression determining offset count.
- Indexed Variable: The entire term `sicaklik[n + 2]` references the specific memory cell.
- Assignment Value (`= 32`): The right-hand value stored into the calculated address.

Base Name: `sicaklik`

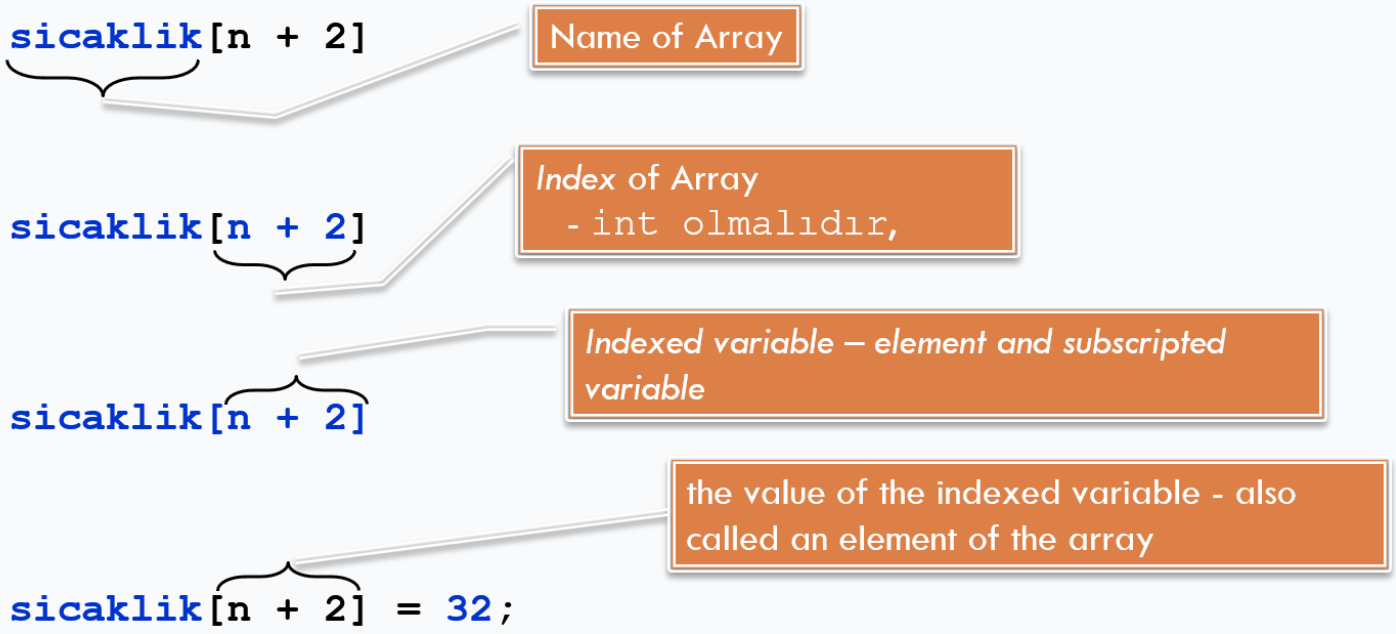
Identifies starting RAM memory address anchor

Index Expression: `n + 2`

Evaluates to an integer offset (e.g. 3 + 2 = 5)

Cell Assignment: `= 32`

Stores value 32 into memory location Base + (5 * size)



Full C++ Code Example: Traversal Loop

Declaring & Printing Array

- Header Inclusion: `#include <iostream>` for C++ stream I/O.
- Array Declaration: `int numbers[5] = {10, 20, 30, 40, 50};`.
- Loop Iteration: `for (int i = 0; i < 5; i++)` traverses indexes 0 to 4.
- Output Stream: `std::cout` prints element at index i.
- Expected Output: `10 20 30 40 50`.

```cpp
// C++ Traversal Program
#include <iostream>
using namespace std;

int main() {
    // Declare and initialize array
    int numbers[5] = {10, 20, 30, 40, 50};

    // Access and print elements using loop
    for (int i = 0; i < 5; i++) {
        cout << "Element at index " << i
             << ": " << numbers[i] << endl;
    }
    return 0;
}
```

```
Element at index 0: 10
Element at index 1: 20
Element at index 2: 30
Element at index 3: 40
Element at index 4: 50
```

Advantages & Disadvantages of Arrays

Key Advantages

- Fast O(1) Access: Direct random lookup via index offset in constant time.
- Efficient Memory: Contiguous RAM allocation has zero pointer storage overhead.
- Cache Friendly: Sequential memory arrangement maximizes CPU spatial cache hits.
- Simple Implementation: Easy to declare, manipulate, and pass to functions.

Disadvantages & Limits

- Fixed Size: Capacity cannot be expanded or shrunk dynamically after creation.
- Costly Insert/Delete: Requires shifting elements yielding O(N) worst-case time.
- Memory Waste: Allocating max capacity for small datasets wastes unused RAM.
- Contiguous RAM Need: Allocation fails if RAM lacks a large uninterrupted block.

When to Use & When NOT to Use Arrays

When to Use Arrays

- Fixed Dataset Size: Total record count is known in advance and remains constant.
- Frequent Random Lookup: Application requires frequent reads by numeric index.
- Sequential Iteration: Processing all items from start to finish in loops.
- Multi-Dimensional Grids: Representing matrices, image pixels, game boards.

When NOT to Use Arrays

- Dynamic Growth: Dataset size fluctuates heavily during runtime execution.
- Frequent Middle Inserts: Continually inserting/deleting elements in sorted order.
- Unknown Initial Size: Capacity cannot be estimated before program launch.
- Better Alternatives: Dynamic Vectors (`std::vector`) or Linked Lists.

Overview of 8 Core Array Operations

Fundamental Operations Supported by Arrays

- 1. Traversing: Visiting and processing each element sequentially.
- 2. Insertion: Adding an element at a specified index (requires right shift).
- 3. Deletion: Removing an element at a specified index (requires left shift).
- 4. Searching: Finding the index of a target key (Linear Search O(N)).
- 5. Sorting: Ordering elements in ascending/descending sequence (Bubble Sort O(N^2)).
- 6. Updating: Modifying value stored at a valid index (O(1)).
- 7. Reversing: Inverting array element order in-place.
- 8. Merging: Combining two arrays into a third destination array.

Read / Lookup (O(1))

Traversing, Updating, Direct Index Access

Search & Sort

Linear Search O(N), Bubble Sort O(N^2)

Structural Mutation

Insertion O(N), Deletion O(N), Reversing O(N), Merging O(N+M)

Operation 1: Traversing Array Elements

Traversing Algorithm & Pseudocode

- Purpose: Visit and print/process each element in the array sequentially.
- Time Complexity: O(N) linear time (processes all N elements once).
- Pseudocode:
- Algorithm TraverseArray(A, n)
- For i <- 0 to n-1 do
- Print A[i]
- End For
- End Algorithm

```cpp
// C++ Traversal Code
#include <iostream>
using namespace std;

void traverseArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    traverseArray(arr, 5);
    return 0; // Output: 10 20 30 40 50
}
```

Operation 2: Insertion (Adding an Element)

Insertion Mechanism & Pseudocode

- Purpose: Insert a new element at a target index `pos`.
- Requirement: Elements from `n-1` down to `pos` must shift RIGHT.
- Time Complexity: O(N) worst-case (inserting at index 0 shifts all N items).
- Pseudocode:
- Algorithm InsertElement(A, n, pos, val)
- For i <- n-1 down to pos do A[i+1] <- A[i]
- A[pos] <- val; n <- n + 1
- End Algorithm

```
// C++ Insertion Code
#include <iostream>
using namespace std;

int main() {
    int arr[6] = {10, 20, 30, 40, 50};
    int n = 5, pos = 2, newVal = 25;

    // Shift elements right
    for (int i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }
    arr[pos] = newVal; // Insert
    n++;               // Increment size

    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    return 0; // Output: 10 20 25 30 40 50
}
```

Operation 3: Deletion (Removing an Element)

Deletion Mechanism & Pseudocode

- Purpose: Delete an element from a specific target index `pos`.
- Requirement: Elements from `pos+1` up to `n-1` must shift LEFT.
- Time Complexity: O(N) worst-case (deleting index 0 shifts N-1 items).
- Pseudocode:
- Algorithm DeleteElement(A, n, pos)
- For i <- pos to n-2 do A[i] <- A[i+1]
- n <- n - 1
- End Algorithm

```
// C++ Deletion Code
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int n = 5, pos = 2; // Delete index 2 (value 30)

    // Shift elements left
    for (int i = pos; i < n - 1; i++) {
        arr[i] = arr[i + 1];
    }
    n--; // Decrement size

    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    return 0; // Output: 10 20 40 50
}
```

Operation 4: Searching (Linear Search)

Linear Search Concept & Algorithm

- Purpose: Find the index of a target `key` by checking items sequentially.
- Best Case: O(1) if target key is at index 0.
- Worst Case: O(N) if target key is at last index or missing.
- Pseudocode:
- Algorithm LinearSearch(A, n, key)
- For i <- 0 to n-1 do
- If A[i] == key then Return i
- End For
- Return -1 // Not found

```cpp
// C++ Linear Search Code
#include <iostream>
using namespace std;

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) return i; // Found
    }
    return -1; // Not found
}

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int index = linearSearch(arr, 5, 30);
    cout << "Index: " << index << endl; // Output: 2
    return 0;
}
```

Operation 5: Sorting (Bubble Sort Pseudocode)

Bubble Sort Mechanics

- Concept: Repeatedly compares adjacent elements `A[j]` and `A[j+1]` and swaps them if out of order.
- Bubbling Effect: The largest unsorted element 'bubbles up' to its correct final position in each pass.
- Passes Required: `n - 1` outer passes.
- Time Complexity: O(N^2) quadratic time.

Algorithm BubbleSort(A, n)

Input: Unsorted Array A, Size n

Outer Loop: For i <- 0 to n-2

Controls total number of sorting passes

Inner Loop: For j <- 0 to n-i-2

Compares adjacent pairs and swaps if A[j] > A[j+1]

Operation 5: Sorting (Bubble Sort C++ Code)

Bubble Sort Implementation

- Nested Loops: Outer loop controls pass index `i`; inner loop compares adjacent `j` and `j+1`.
- Swap Logic: Uses temporary variable `temp` to exchange values.
- Optimization: Inner loop bound `n - i - 1` avoids re-checking already sorted tail elements.
- Output: `10 20 30 40 50`.

```cpp
// C++ Bubble Sort Code
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[5] = {50, 20, 40, 10, 30};
    bubbleSort(arr, 5);
    for (int i = 0; i < 5; i++) cout << arr[i] << " ";
    return 0; // Output: 10 20 30 40 50
}
```

Operation 6: Updating (Modifying an Element)

Direct Memory Update Concept

- Purpose: Modify the value stored at a known valid index `index`.
- Time Complexity: O(1) constant time (direct access via address calculation).
- No Shifting Required: Does not alter array size or shift adjacent elements.
- Pseudocode:
- Algorithm UpdateElement(A, index, newValue)
- A[index] <- newValue
- End Algorithm

```
// C++ Update Code
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    // Update element at index 2 (replace 30 with 35)
    arr[2] = 35;

    for (int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
    }
    return 0; // Output: 10 20 35 40 50
}
```

Operation 7: Reversing (Inverting the Array)

Two-Pointer Reversing Strategy

- Purpose: Invert array element order in-place (first becomes last, second becomes second-last).
- Two-Pointer Technique: Swap `arr[i]` with `arr[n - i - 1]` up to middle `n / 2`.
- Time Complexity: O(N) linear time.
- Space Complexity: O(1) auxiliary space (in-place modification).

```cpp
// C++ Reversing Code
#include <iostream>
using namespace std;

void reverseArray(int arr[], int n) {
    for (int i = 0; i < n / 2; i++) {
        int temp = arr[i];
        arr[i] = arr[n - i - 1];
        arr[n - i - 1] = temp;
    }
}

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    reverseArray(arr, 5);
    for (int i = 0; i < 5; i++) cout << arr[i] << " ";
    return 0; // Output: 50 40 30 20 10
}
```

Operation 8: Merging Two Sorted Arrays (Pseudocode)

Merging Algorithm Concept

- Purpose: Combine two sorted input arrays `A` (size n1) and `B` (size n2) into a single sorted array `C` (size n1 + n2).
- Two Pointers `i` and `j`: Compare current items `A[i]` and `B[j]`, copy the smaller item into `C[k]`, and advance pointers.
- Cleanup Loops: Copy any remaining items from `A` or `B` after one array is exhausted.
- Time Complexity: O(n1 + n2) linear time.

Compare & Copy

While i < n1 and j < n2: Copy min(A[i], B[j]) to C[k]

Exhaust A

While i < n1: Copy remaining elements of A to C[k]

Exhaust B

While j < n2: Copy remaining elements of B to C[k]

Operation 8: Merging Two Sorted Arrays (C++ Code)

C++ Merge Implementation

- Pointer Tracking: `i` for `arr1`, `j` for `arr2`, `k` for `result`.
- Comparison Loop: `while (i < n1 && j < n2)` appends smaller element.
- Residual Tail Cleanup: Two trailing `while` loops flush remaining items.
- Output: `10 15 20 25 30 35`.

```cpp
// C++ Merge Function
#include <iostream>
using namespace std;

void mergeArrays(int a1[], int n1, int a2[], int n2, int res[]) {
    int i = 0, j = 0, k = 0;
    while (i < n1 && j < n2) {
        if (a1[i] < a2[j]) res[k++] = a1[i++];
        else               res[k++] = a2[j++];
    }
    while (i < n1) res[k++] = a1[i++];
    while (j < n2) res[k++] = a2[j++];
}

int main() {
    int a1[3] = {10, 20, 30}, a2[3] = {15, 25, 35}, res[6];
    mergeArrays(a1, 3, a2, 3, res);
    for (int x : res) cout << x << " ";
    return 0; // Output: 10 15 20 25 30 35
}
```

Example 1: Finding Sum of Array Elements

Accumulator Pattern Pseudocode

- Input: Array `A` of size `n`.
- Output: Total sum of all integer elements.
- Accumulator Variable: Initialize `toplam <- 0` before loop.
- Traversal Loop: `FOR i <- 0 TO n-1 DO toplam <- toplam + A[i]`.
- Return Value: Return `toplam`.

Step 1: Initialize

Set accumulator variable `toplam = 0`

Step 2: Traverse

Loop i from 0 to n-1 and execute `toplam += A[i]`

Step 3: Return

Return accumulated total in O(N) time

Example 1: Sum & Average Calculation in C++

Sum & Average Implementation

- Initialization: `Total = 0` and `counter = 0` MUST be initialized to 0.
- Element Count: Calculated dynamically via `sizeof(Numbers) / sizeof(int)`.
- Accumulation Loop: `while (counter < ElementCount)` accumulates `Total += Numbers[counter]`.
- Average Calculation: Requires casting `Total` to floating point to avoid integer truncation.

```cpp
// C++ Sum Calculation
#include <iostream>
using namespace std;

int main() {
    int Numbers[5] = {11, 5, 23, 4, 54};
    int counter = 0, Total = 0;
    int ElementCount = sizeof(Numbers) / sizeof(int);

    while (counter < ElementCount) {
        Total += Numbers[counter];
        counter++;
    }

    cout << "Total Value = " << Total << endl;
    cout << "Average = " << (double)Total / ElementCount << endl;
    return 0;
}
```

Example 2: Finding Largest Element (Max Search)

Max Search Logic

- Initial Assumption: Assume first element is maximum (`Maximum = Numbers[0]`).
- Comparison Loop: Iterate through remaining elements.
- Update Condition: If `Numbers[counter] > Maximum`, update `Maximum = Numbers[counter]`.
- Time Complexity: O(N) linear scan.
- Exercise Challenge: Rewrite using a `for` loop instead of `while` loop!

```cpp
// C++ Max Search Code
#include <iostream>
using namespace std;

int main() {
    int Numbers[10] = {5, 55, 49, 87, 63, 18, 72, 26, 45, 74};
    int counter = 1;
    int Maximum = Numbers[0]; // Assume index 0 is max

    while (counter < 10) {
        if (Numbers[counter] > Maximum) {
            Maximum = Numbers[counter]; // Update max
        }
        counter++;
    }

    cout << "Maximum Value = " << Maximum << endl;
    return 0; // Output: 87
}
```

Example 3: Reading, Sorting & Printing Numbers

3-Phase Program Anatomy

- Phase 1: Input Loop (`cin >> array[i]`) reads 10 numbers from keyboard into array.
- Phase 2: Sorting Loop (Nested `for` loops) executes Bubble Sort algorithm.
- Phase 3: Output Loop (`cout << array[d]`) displays sorted array on screen.
- Modularity: Shows complete workflow from raw user input to sorted display.

```
// C++ Complete Sorting Code
#include <iostream>
using namespace std;

int main() {
    int array[10], temp;
    // Phase 1: Input 10 numbers
    for (int i = 0; i <= 9; i++) {
        cout << "Enter num " << i+1 << ": "; cin >> array[i];
    }
    // Phase 2: Bubble Sort
    for (int a = 0; a <= 9; a++) {
        for (int b = 0; b <= 8; b++) {
            if (array[b] > array[b + 1]) {
                temp = array[b]; array[b] = array[b + 1]; array[b +
1] = temp;
            }
        }
    }
    // Phase 3: Output sorted array
    for (int d = 0; d <= 9; d++) cout << array[d] << " ";
    return 0;
}
```

Example 3: Bubble Sort Algorithmic Flowchart

Flowchart Control Flow Structure

- Start Node: Initialize outer loop index `a = 0`.
- Outer Loop Condition (`a <= 9`): Controls total passes; terminates when `a > 9`.
- Inner Loop Initialization (`b = 0`): Resets inner loop for pass.
- Inner Loop Condition (`b <= 8`): Checks adjacent pairs.
- Decision Box (`dizi[b] > dizi[b+1]`): If True (E), execute Swap (`temp = dizi[b]...`); If False (H), advance `b = b + 1`.

Outer Loop `a`

Iterates passes from 0 to 9

Inner Loop `b`

Compares adjacent pairs b and b+1

Swap Decision

Exchanges elements if out of order

Flowchart Execution Trace: Pass 1 Setup

Initial State & Variables

- Target Array: `dizi = [18, 8, 21, 16, 13, 5, 6, 22, 26, 30]` (Size = 10).
- Outer Loop Counter: `a = 0`.
- Inner Loop Counter: `b = 0`.
- Current Pair Compared: `dizi[0] = 18` and `dizi[1] = 8`.
- Condition Check: `18 > 8` evaluates to TRUE (E).

Before Swap

`dizi[0]=18`, `dizi[1]=8` -> Condition `18 > 8` is TRUE

Swap Execution

`temp=18; dizi[0]=8; dizi[1]=18;`

After Swap Array

`dizi = [8, 18, 21, 16, 13, 5, 6, 22, 26, 30]`

Flowchart Execution Trace: Pass 1 Continuing

Advancing Inner Loop `b=1` and `b=2`

- Increment Inner Counter: `b` becomes `1`.
- Pair Check 1 (`b=1`): Compare `dizi[1] = 18` and `dizi[2] = 21`. `18 > 21` is FALSE (H) -> No Swap! Increment `b=2`.
- Pair Check 2 (`b=2`): Compare `dizi[2] = 21` and `dizi[3] = 16`. `21 > 16` is TRUE (E) -> Swap Executed!
- Resulting Array: `dizi = [8, 18, 16, 21, 13, 5, 6, 22, 26, 30]`.

b=1 Check

18 > 21 is FALSE -> No Swap, increment b to 2

b=2 Check

21 > 16 is TRUE -> Swap 21 and 16

Updated Array

`dizi = [8, 18, 16, 21, 13, 5, 6, 22, 26, 30]`

Example 4: Linear Search & Index Return

Searching Array Function

- Function Signature: `int dogrusalArama(int A[], int N, int Aranan);`.
- Loop Traversal: `for (int K=0; K<N; K++)` checks each element.
- Match Found: `if (A[K] == Aranan) return K;` returns index offset.
- Not Found: Returns `-1` after full traversal.
- Main Routine: Reads target search number `Ara` from user and displays index result.

```cpp
// C++ Linear Search Code
#include <iostream>
using namespace std;

int dogrusalArama(int A[], int N, int Aranan) {
    for (int K = 0; K < N; K++) {
        if (A[K] == Aranan) return K; // Return index
    }
    return -1; // Not found
}

int main() {
    int B[10] = {13, 12, 15, 11, 26, 5, 47, 10, 27, 34};
    int Ara = 26;
    int indis = dogrusalArama(B, 10, Ara);
    if (indis == -1) cout << "Not found!";
    else cout << "Found at index: " << indis;
    return 0; // Output: Found at index: 4
}
```

Example 5: Soft Deletion Using Sentinel (-1 Flag)

Logical Deletion without Element Shifting

- Concept: Instead of shifting elements, overwrite deleted item with a sentinel value `-1`.
- Positive Integer Assumption: Works when all valid data items are positive integers.
- Fast Deletion O(1) after search: `B[indis] = -1;` soft-deletes in constant time.
- Limitation: Leaves 'deleted holes' in memory array structure.

```cpp
// C++ Soft Deletion Code
#include <iostream>
using namespace std;

int dogrusalArama(int A[], int N, int Aranan) {
    for (int K = 0; K < N; K++) if (A[K] == Aranan) return K;
    return -1;
}

int main() {
    int B[10] = {13, 12, 15, 11, 26, 5, 47, 10, 27, 34};
    int Ara = 15;
    int indis = dogrusalArama(B, 10, Ara);
    if (indis != -1) {
        B[indis] = -1; // Mark as deleted (sentinel)
        cout << "Element " << Ara << " deleted at index " << indis;
    }
    return 0;
}
```

Example 6: Reusing Empty Slots (-1 Flag)

Inserting into First Empty Slot

- Concept: Search for the first sentinel slot `-1` and store new element there.
- Search Target: `indis = dogrusalArama(B, 10, -1);`.
- Insertion: `B[indis] = Eklenen;` fills the vacated slot.
- Efficiency: Avoids expanding array capacity when deleted slots exist.

```
// C++ Reuse Empty Slot Code

#include <iostream>
using namespace std;

int dogrusalArama(int A[], int N, int Aranan) {
    for (int K = 0; K < N; K++) if (A[K] == Aranan) return K;
    return -1;
}

int main() {
    int B[10] = {13, 12, -1, 11, 26, 5, 47, -1, 27, 34};
    int Eklenen = 99;

    // Search for first empty space marked by -1
    int indis = dogrusalArama(B, 10, -1);
    if (indis != -1) {
        B[indis] = Eklenen; // Insert into slot
        cout << "Inserted " << Eklenen << " at index " << indis;
    }
    return 0; // Inserts 99 at index 2
}
```

Example 7: Array Structure with `struct`

Defining User-Defined Record Type

- Composite Record: `struct Ogrenci_Bilgisi` groups student attributes.
- Fields: `numara`, `adi`, `soyadi`, `ara_sinav_notu`, `final_sinav_notu`, `odev_notu`.
- Struct Variable: `Ogrenci_Bilgisi ogrenci;` declares a record variable.
- Semicolon Requirement: Must end `struct` declaration with a semicolon `;`.

```
// C++ Struct Declaration
#include <iostream>
#include <string>
using namespace std;

// Struct definition
struct Ogrenci_Bilgisi {
    int numara;
    string adi;
    string soyadi;
    float ara_sinav_notu;
    float final_sinav_notu;
    float odev_notu;
}; // Semicolon required!

int main() {
    Ogrenci_Bilgisi ogrenci;
    ogrenci.numara = 2316;
    ogrenci.adi = "Omer";
    cout << "Student: " << ogrenci.adi << " (" << ogrenci.numara <<
")\n";
    return 0;
}
```

Example 8: Struct Array Population in Loop

Processing Array of Composite Records

- Struct Array Declaration: `Ogrenci_Bilgisi ogrenciler[5];` allocates memory for 5 student records.
- Loop Iteration: `for (i=0; i<5; i++)` reads student attributes sequentially.
- Field Access: Dot operator combined with index `ogrenciler[i].numara`.
- Real-World Pattern: Standard pattern for memory-based database tables.

```
// C++ Struct Array Loop Code
#include <iostream>
#include <string>
using namespace std;

struct Ogrenci_Bilgisi {
    int numara; string adi; string soyadi;
    float ara_sinav_notu; float final_sinav_notu; float odev_notu;
};

int main() {
    Ogrenci_Bilgisi ogrenciler[5];
    for (int i = 0; i < 5; i++) {
        cout << "Enter student " << i + 1 << " ID: ";
        cin >> ogrenciler[i].numara;
        cout << "Enter name: ";
        cin >> ogrenciler[i].adi;
        // Read remaining fields...
    }
    return 0;
}
```

Homework 1: Phonebook Application in C++

Project Requirements

- Capacity: Design application to store a maximum of 50 phone records.
- Record Attributes: Record number, Name, Surname, Home Phone, Mobile Phone, Email.
- Supported Operations: Add record, Search record, List all records, Delete record.
- Data Structure: Array of `struct PhoneRecord`.

Deliverables & Submission

- 1. Pseudocode: Detailed pseudocode for all subcomponents.
- 2. Flowchart: Complete visual flowchart diagrams for CRUD functions.
- 3. C++ Source Code: Clean, commented C++ implementation file.
- 4. Testing Report: Terminal screenshots verifying insert, search, list, and delete.